GraphCompose

v2.2.0

twinned

Open-source Java library for generating structured business documents with a declarative DSL — print-ready PDF and an editable PowerPoint deck from one composition.

1 Java · Declarative DSL

```
var out = Path.of("deck.pdf");
try (var doc = GraphCompose
.document(out)
.pageSize(DocumentPageSize.A4)
.create()) {
doc.pageFlow(page -> page
.addParagraph("GraphCompose")
.addParagraph("Cinematic."));
doc.buildPdf();
}
```

You describe the document intent — the engine resolves the rest.

2 GraphCompose Engine

Semantic graph · deterministic layout

- Layout
- Components
- Pagination
- Snapshot Tests
- Themes
- DSL

3 Output Backends

- **PDFBox 3.0** — Production backend
- **PPTX** — Editable deck · Beta
- **DOCX export** — Semantic export

4 Real PDF Output

CVs, invoices, reports — every one rendered by the engine in this repo.

Open Source · Maven Central · Java 17+ · MIT License

HOW IT WORKS

From one Java file to a designed document

You describe the document semantically — **sections, rows, tables, charts, shapes, layers** — and the engine resolves the rest: it **measures** every node twice, **paginates** the flow row-by-row, and **renders** the resolved layout through an isolated backend — PDFBox for PDF, Apache POI for the deck. No manual coordinates, no XML templates.

1

AUTHOR

Fluent DSL describes intent.

>

2

MEASURE

Two-pass geometry, every node.

>

3

PAGINATE

Split the flow across pages.

>

4

RENDER

A backend writes the bytes.

Deterministic

The same input renders the same bytes on every machine — layout is reproducible, not best-effort.

Regression-tested

layoutSnapshot() diffs the geometry and PdfVisualRegression pixel-tests fonts and colour, right in your pull requests.

Production-ready

An isolated PDFBox backend streams straight to an HTTP response — no temp files, no manual coordinates.

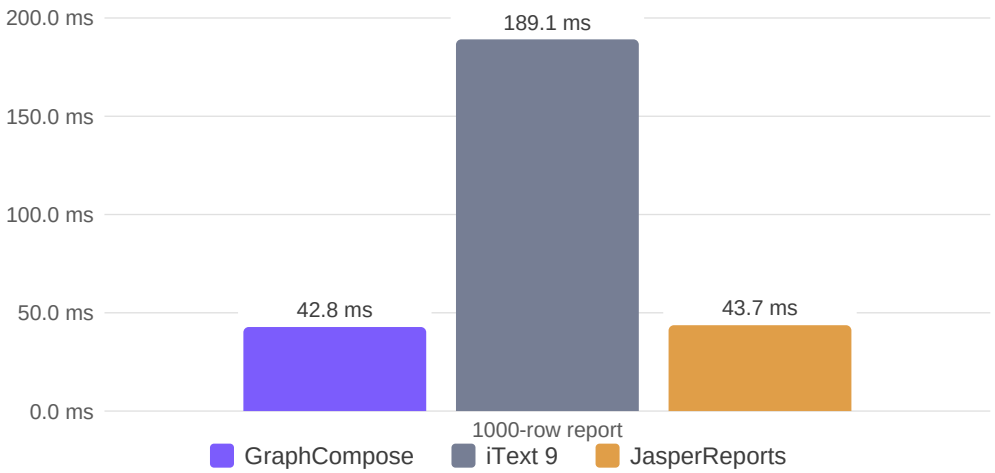
BENCHMARKED

GraphCompose vs. the field

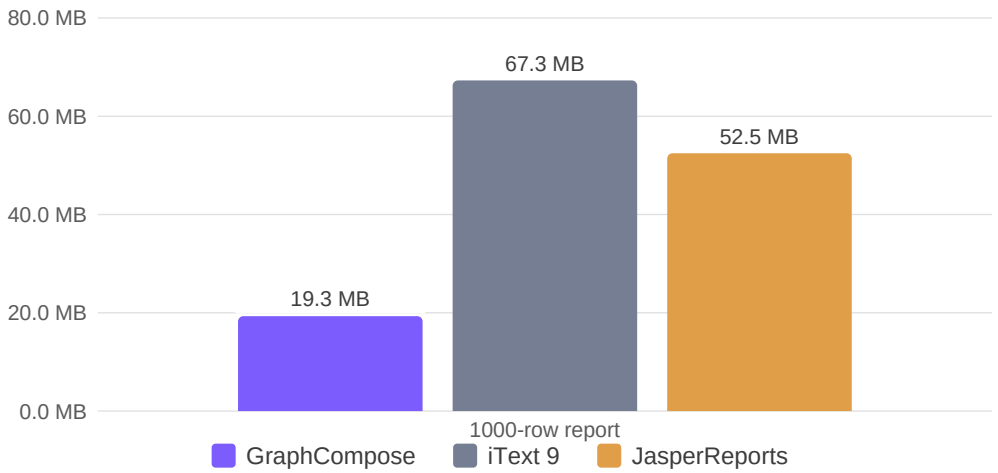
The same documents through three engines — render time and peak heap, measured by this repository's own harness. The table and charts below are drawn by GraphCompose straight from the result file.

Report size	GraphCompose	iText 9	JasperReports
1 page · 3 lines	2.0 ms · 0.2 MB	2.1 ms · 0.2 MB	4.2 ms · 0.2 MB
40 rows	4.1 ms · 1.0 MB	10.0 ms · 3.0 MB	8.3 ms · 2.5 MB
200 rows	10.9 ms · 4.0 MB	36.6 ms · 13.6 MB	13.5 ms · 10.8 MB
1000 rows	42.8 ms · 19.3 MB	189.1 ms · 67.3 MB	43.7 ms · 52.5 MB

Render time at 1000 rows — ms (lower is faster)



Peak heap at 1000 rows — MB (lower is lighter)



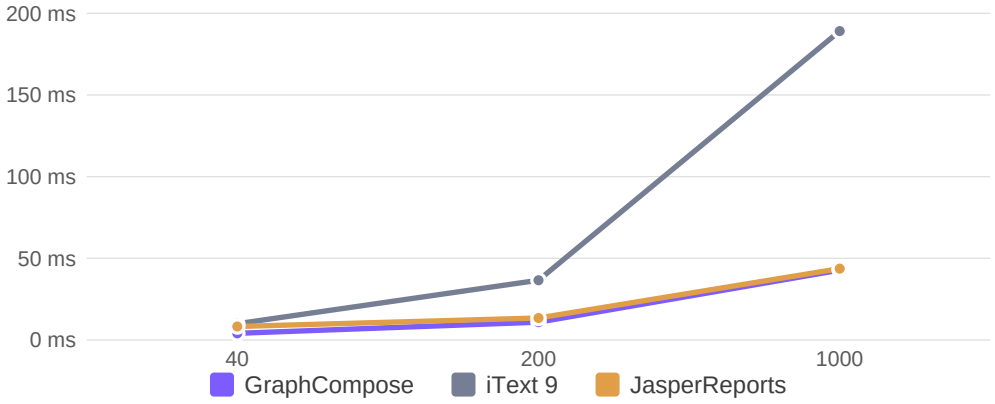
Measured 2026-08-04 22:16:04 · 50 warmup / 100 measurement iterations, single machine — numbers vary by hardware and document. Reproduce: scripts/run-benchmarks.ps1 · see docs/operations/benchmarks.md.

SCALING

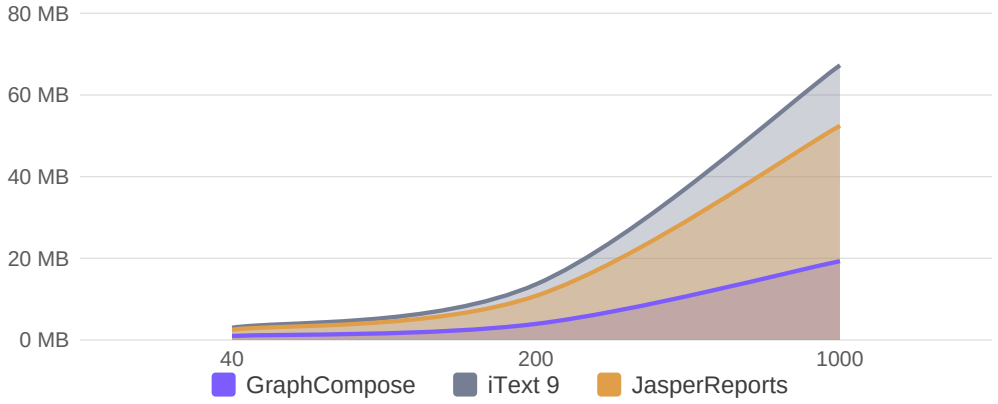
How GraphCompose scales

As the report grows from 40 to 1000 rows, the time lead over iText widens; JasperReports closes to roughly the same render time at 1000 rows, yet GraphCompose stays markedly lighter on memory than both rivals throughout — every series below reads from the same benchmark file.

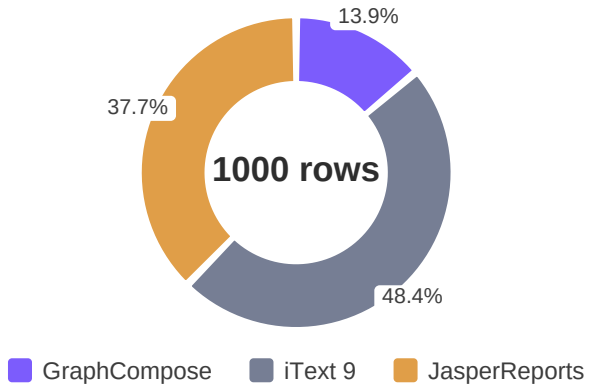
Render time vs. report size — ms



Peak heap vs. report size — MB



Memory at 1000 rows — share of total



Throughput at 1000 rows — docs/sec (higher is better)